Neural Network: Multi-Layer Perceptron

J.H. Lee, School of ICE, Sungkyunkwan Univ.

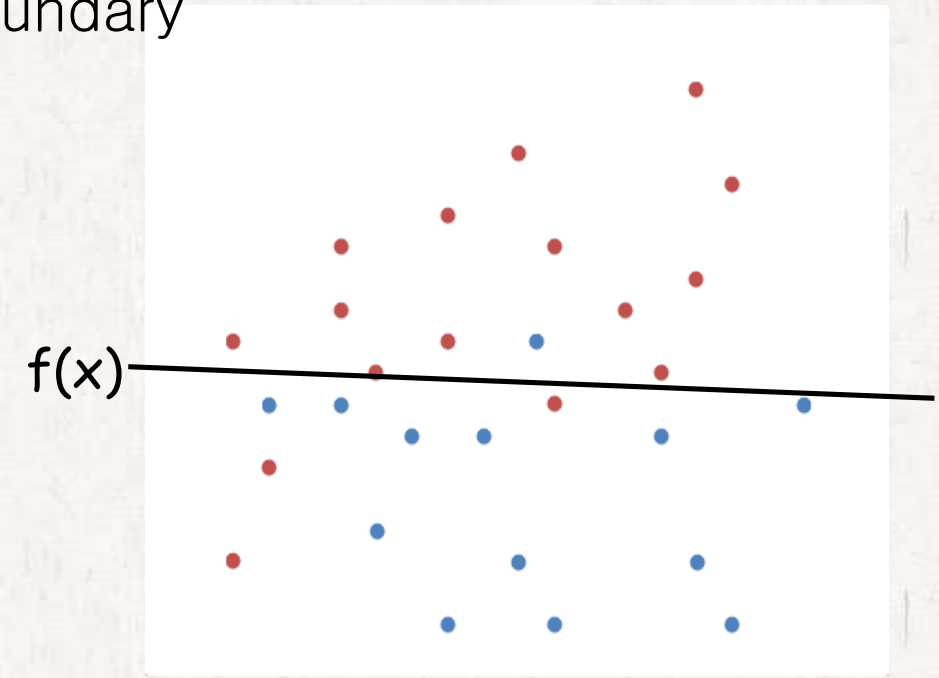
Logistic Regression

- Logistic Regression
  - Binary classification
  - Linear classifier

- Let's extend Logistic Regression
  - Multiple classes
  - Any types of boundary

Logistic Regression

- Linear Boundary

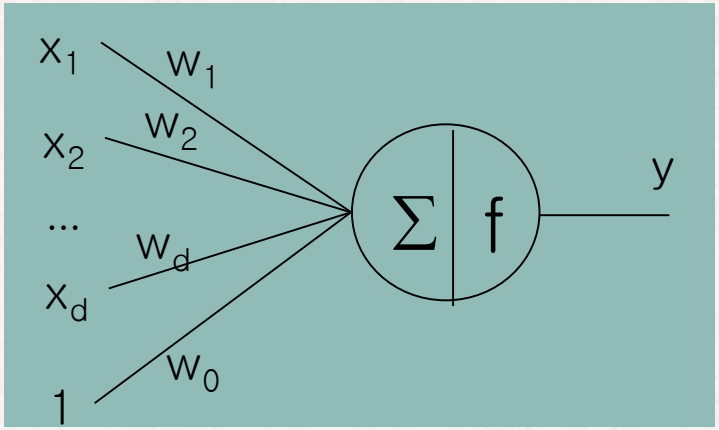


$$\text{Label}(\mathbf{x}) = \begin{cases} 1\,(\text{Red}) & \text{if } h(\mathbf{x}) \geq 0.5 \\ 0\,(\text{Blue}) & \text{if } h(\mathbf{x}) < 0.5 \end{cases} \qquad h(\mathbf{x}) = \frac{1}{1+\exp\left(-f(\mathbf{x})\right)}$$

Logistic Regression

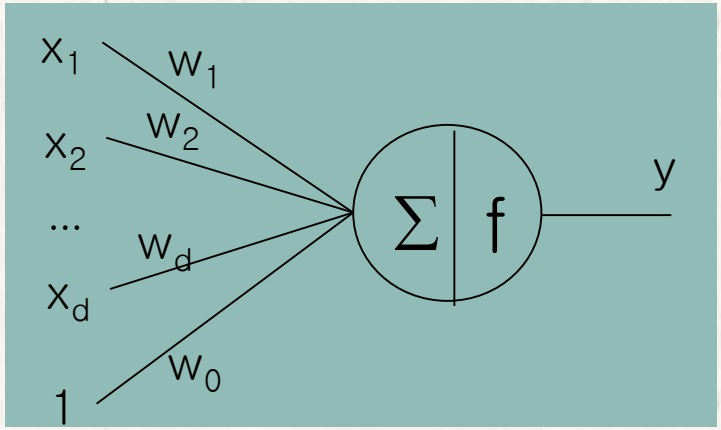
- Two Steps for Evaluation
  - Linear combination of inputs: $s = \mathbf{w}^T \mathbf{x} = \sum_{i=0}^{d} w_i x_i$
  - Nonlinear transform of s: $y = \frac{1}{1+\exp(-s)} = f(s)$

- Graphical Representation



Logistic Regression

- Another Name: Perceptron
  - It may be called an artificial neuron
  - It mimics the function of neurons



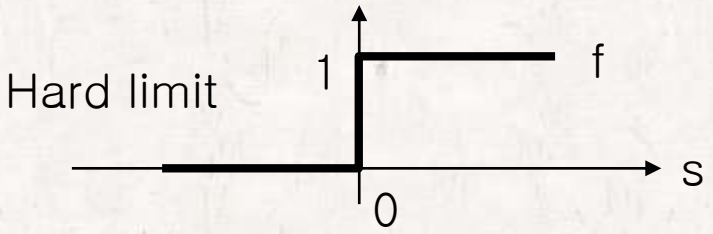
- Logistic Regression is using just ONE perceptron
  - What if we combine many perceptrons

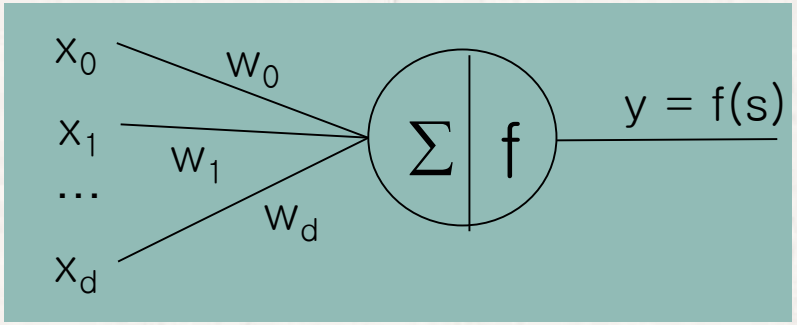
Perceptron

- Perceptron
  - First function: Weighted summation of inputs

$$s = x_0 w_0 + x_1 w_1 + \ldots + x_d w_d$$

  - Second function: Non-linear function

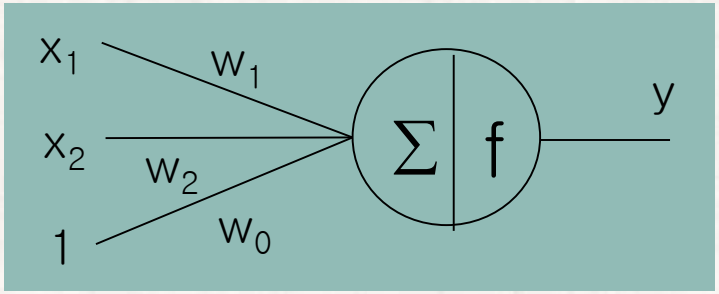




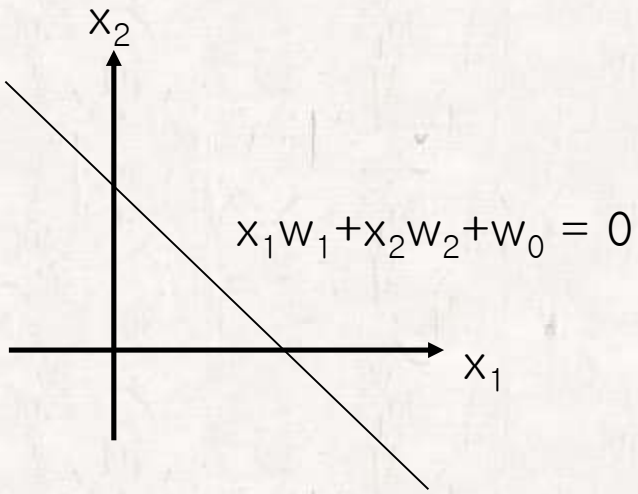
$$y = \begin{cases} 1 & \sum_{i=0}^{n} x_i w_i > 0 \\ 0 & otherwise \end{cases}$$

Perceptron

- What a perceptron does



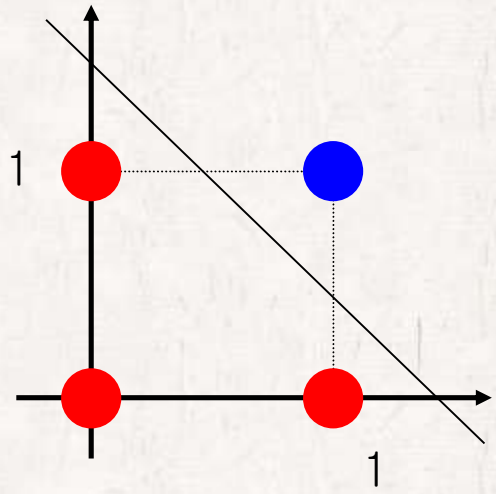
$$y = \begin{cases} 1 & \sum_{i=1}^{n} x_i w_i > 0 \\ 0 & otherwise \end{cases}$$

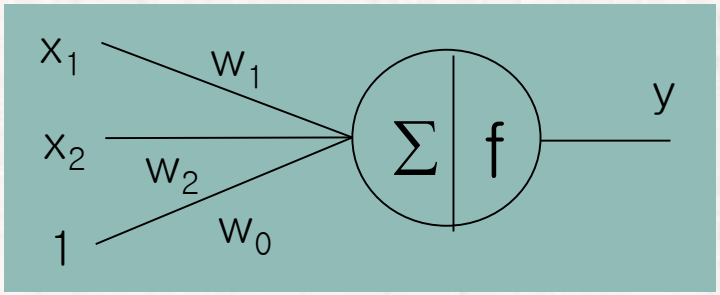


If an input is above the line
output 1
else
output 0

Perceptron

- What a perceptron can do
  - And operation



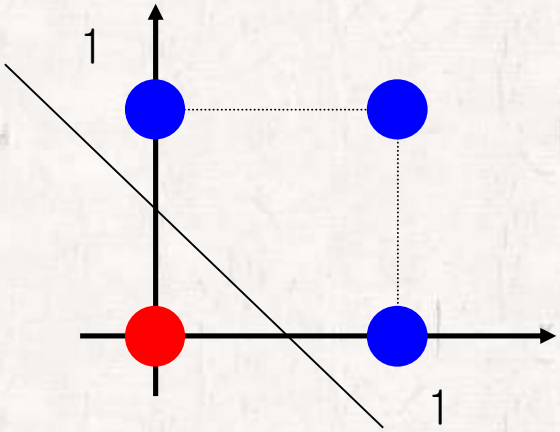


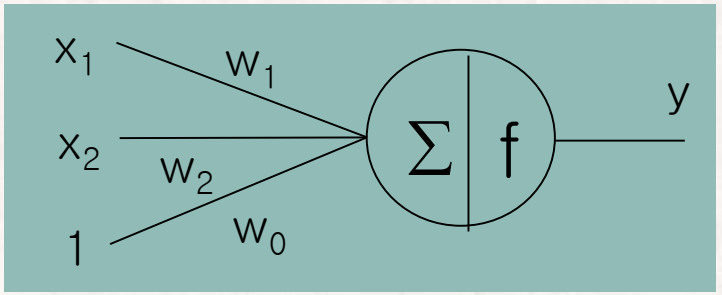
w1=1.0, w2=1.0, w0=−1.5

x_1	x_2	Σ	y
0	0	−1.5	0
0	1	−0.5	0
1	0	−0.5	0
1	1	0.5	1

Perceptron

- What a perceptron can do – con'd
  - OR operation



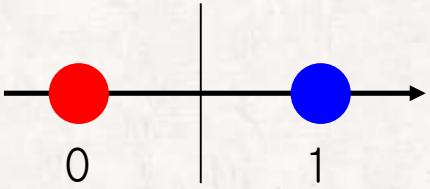


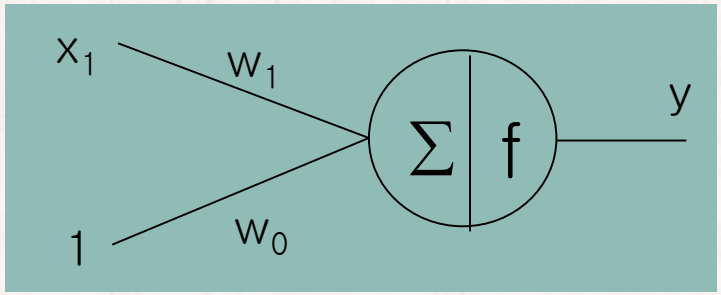
w1=1.0, w2=1.0, w0=−0.5

x_1	x_2	Σ	y
0	0	−0.5	0
0	1	0.5	1
1	0	0.5	1
1	1	1.5	1

Perceptron

- What a perceptron can do – con'd
  - NOT operation



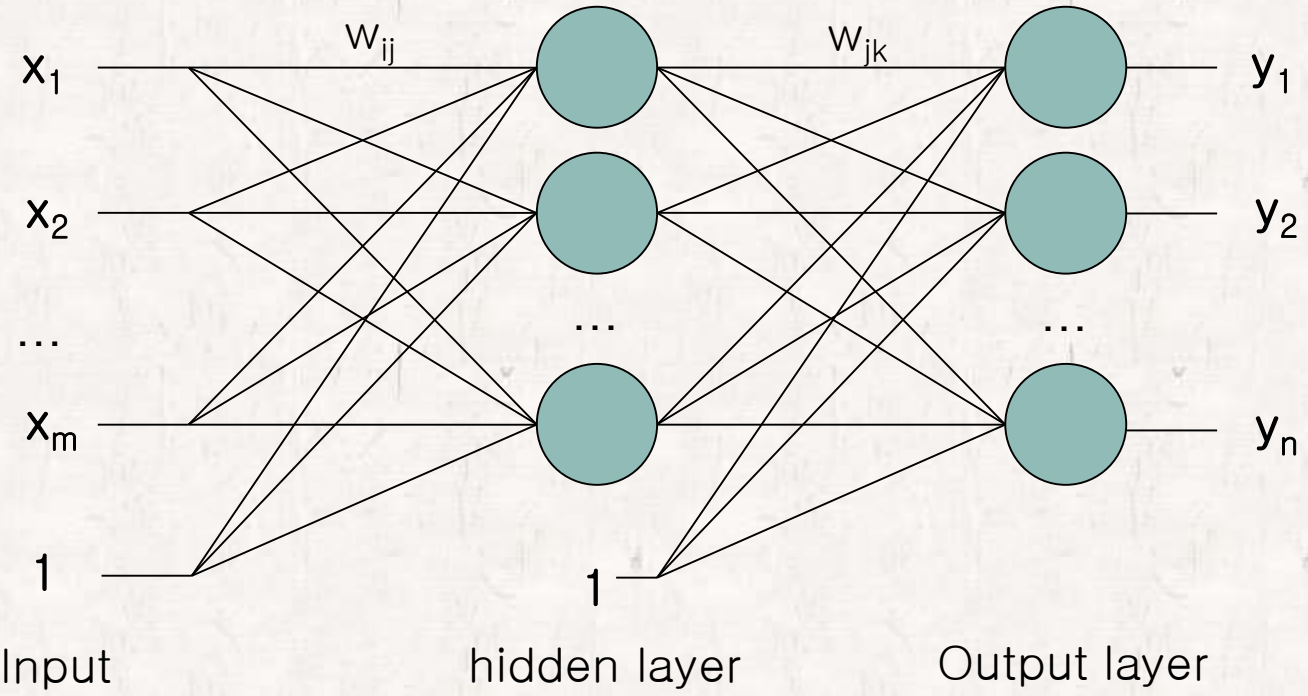


w1=−1.0, w0=0.5

x_1	Σ	y
0	0.5	1
1	−0.5	0

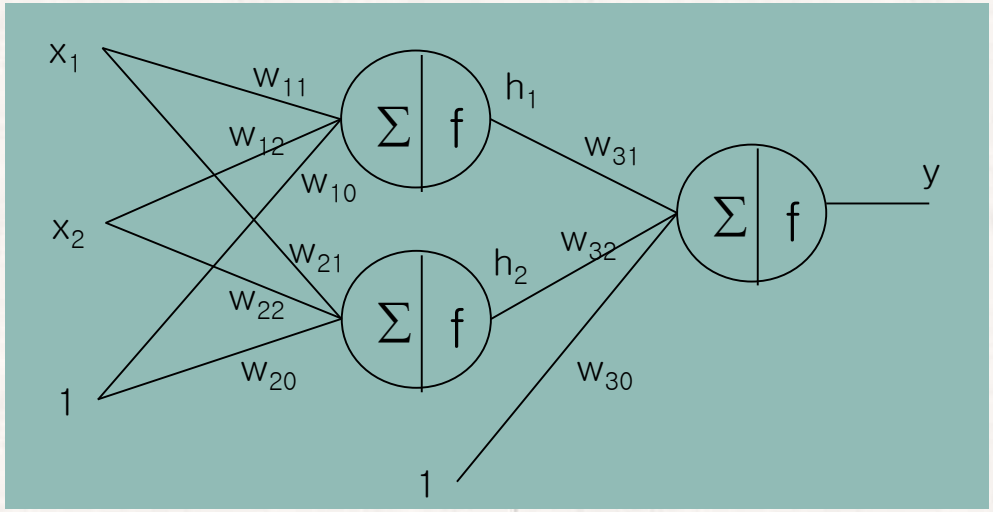
Multilayer Perceptron

- Let's Cascade Many Perceptrons
  - (A network of perceptrons) v.s. (A network of neurons (brain))
  - Layered structures: for simplicity of learning



Multilayer Perceptron

- Graphical Representation is Preferred



$$y = \frac{1}{1+\exp\left(-\left(w_{31}h_1 + w_{32}h_2 + w_{30}\right)\right)}$$

$$h_1 = \frac{1}{1+\exp\left(-\left(w_{11}x_1 + w_{12}x_2 + w_{10}\right)\right)}$$

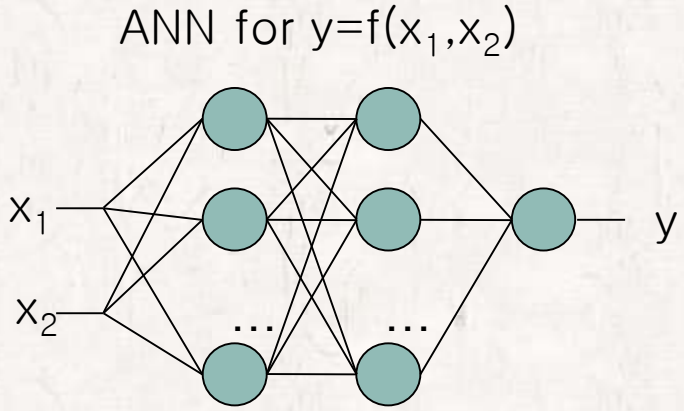
$$h_2 = \frac{1}{1+\exp\left(-\left(w_{21}x_1 + w_{22}x_2 + w_{20}\right)\right)}$$

- Mathematical form of the output

$$y = \frac{1}{1+\exp\left(-\left(w_{31}\left(\frac{1}{1+\exp\left(-\left(w_{11}x_1 + w_{12}x_2 + w_{10}\right)\right)}\right) + w_{32}\left(\frac{1}{1+\exp\left(-\left(w_{21}x_1 + w_{22}x_2 + w_{20}\right)\right)}\right) + w_{30}\right)\right)}$$

Multilayer Perceptron

- Structure of Multilayer Perceptron – con'd
  - Input layer
    - Simply pass the input values to the next layer
    - # of nodes = # of inputs
  - Hidden layer
    - There can be several hidden layers
    - # of nodes should be given
  - Output layer
    - # of nodes = # of outputs



Multilayer Perceptron

- Artificial Neural Network
  - AI tools based on biological brains
  - It can learn anything!!

- It is a type of Artificial Neural Network
  - Multilayer perceptron
  - Kohonen's Self-Organizing Neural Networks
  - ..

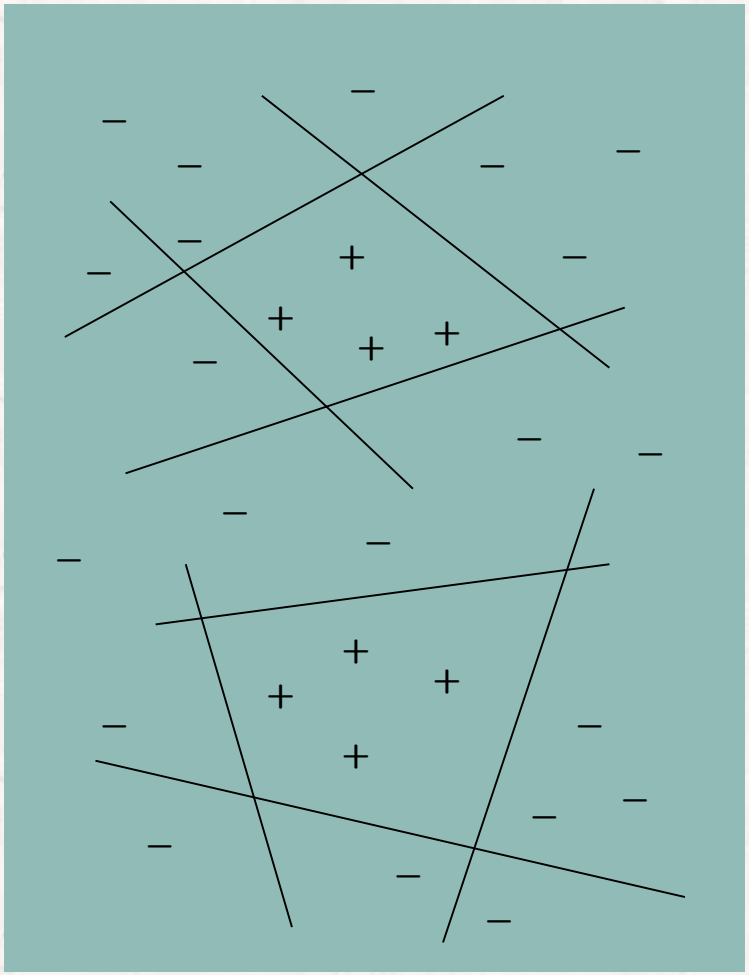
  - Other names of Multilayer Perceptron
  - Feed-forward Neural Network
  - Multilayer Feed-forward Neural Network

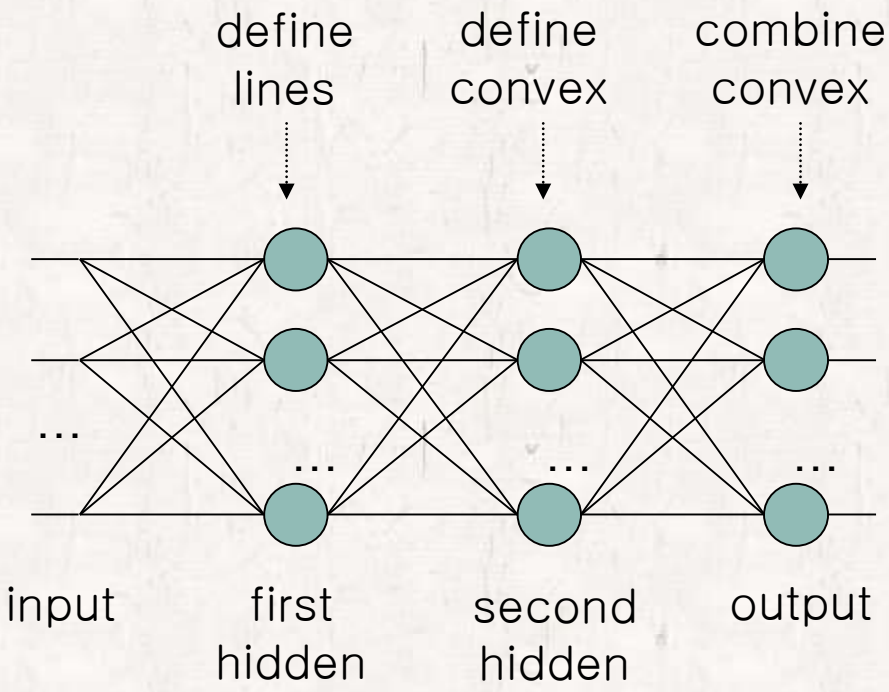
Multilayer Perceptron

- What a multilayer perceptron can do
  - Anything digital computers can do
    - Boolean function: 2-layer perceptron
    - Continuous function: 2-layer perceptron
    - Arbitrary function: 3-layer perceptron
  - We can build a multi-layer perceptron which satisfies given input-output pairs

Multilayer Perceptron

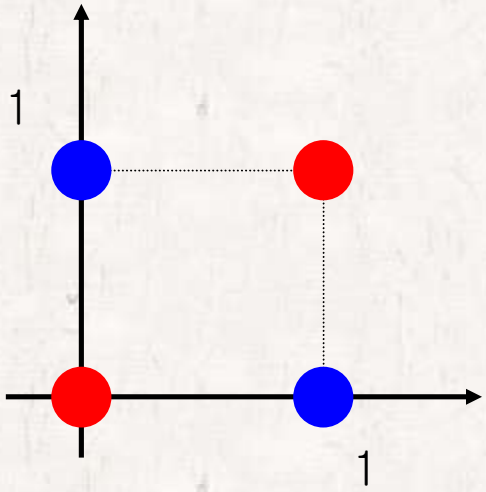
- What a multilayer perceptron can do





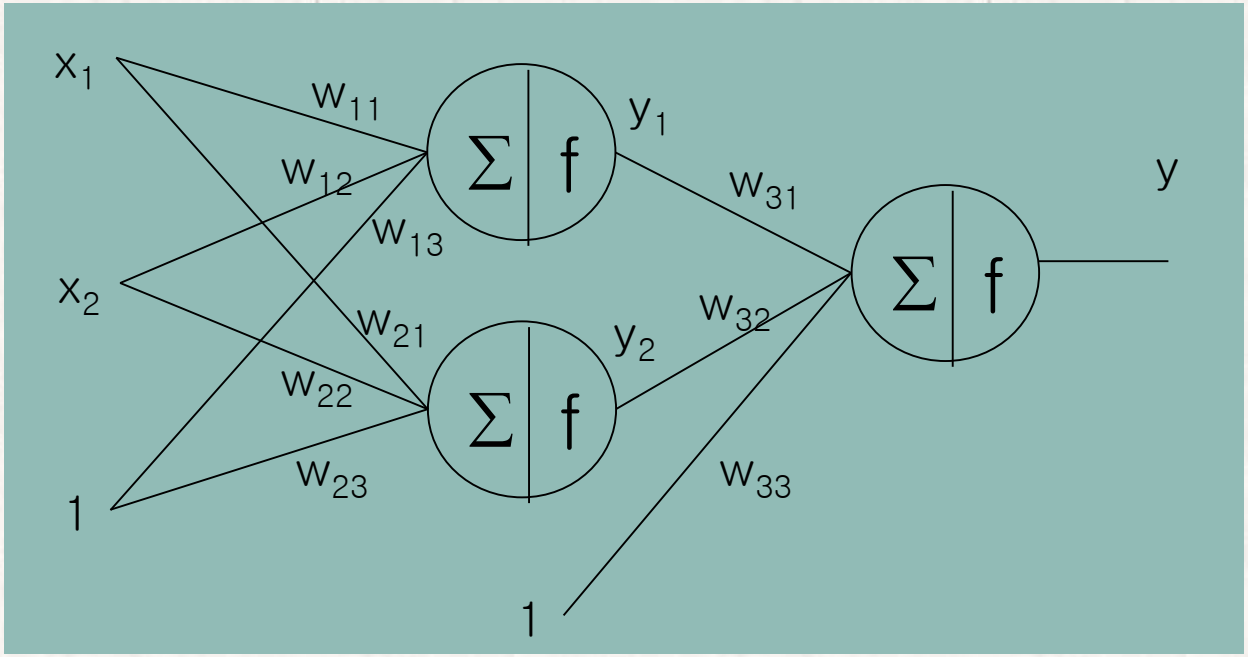
Multilayer Perceptron

- Example: What a neural network can do
  - A neural network can solve non-linearly separable problems
  - Example: XOR operation



Multilayer Perceptron

- Example: What a neural network can do
  - XOR operation



Multilayer Perceptron

- What a neural network can do– con'd
  - XOR operation

$w_{11}=1.0,\ w_{12}=1.0,\ w_{13}=-1.5$

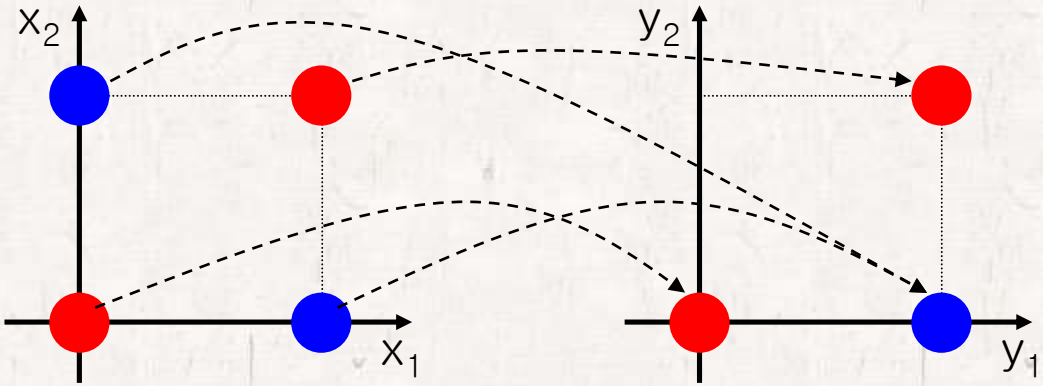
x_1	x_2	Σ	y_1
0	0	−1.5	0
0	1	−0.5	0
1	0	−0.5	0
1	1	0.5	1

$w_{21}=1.0,\ w_{22}=1.0,\ w_{23}=-0.5$

x_1	x_2	Σ	y_2
0	0	−0.5	0
0	1	0.5	1
1	0	0.5	1
1	1	1.5	1

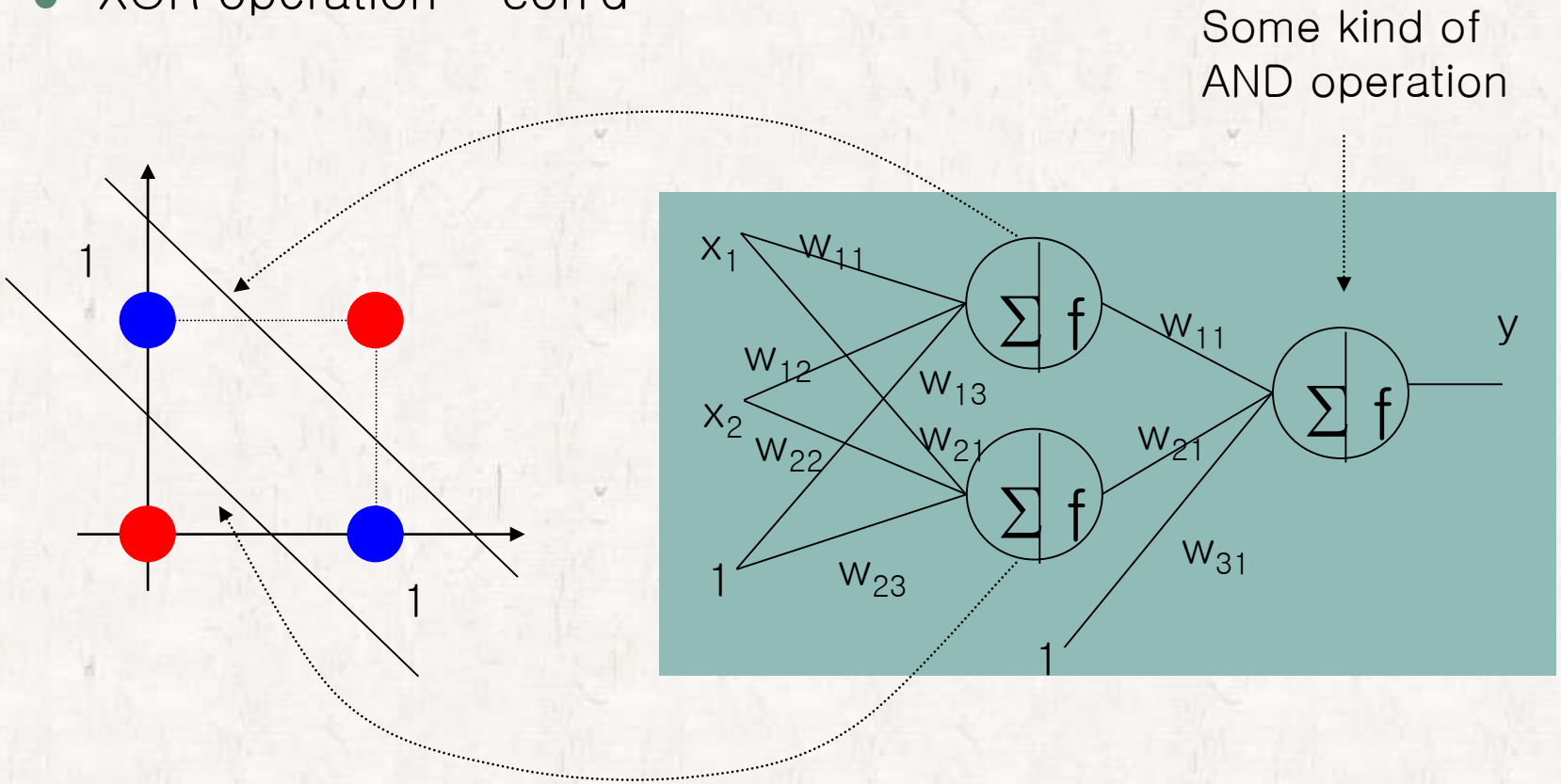
$w_{31}=-1.0,\ w_{32}=1.0,\ w_{33}=-0.5$

y_1	y_2	Σ	y
0	0	−0.5	0
0	1	0.5	1
0	1	0.5	1
1	1	−0.5	0



Multilayer Perceptron

- What a perceptron can do – con'd
  - XOR operation – con'd



Learning Algorithm

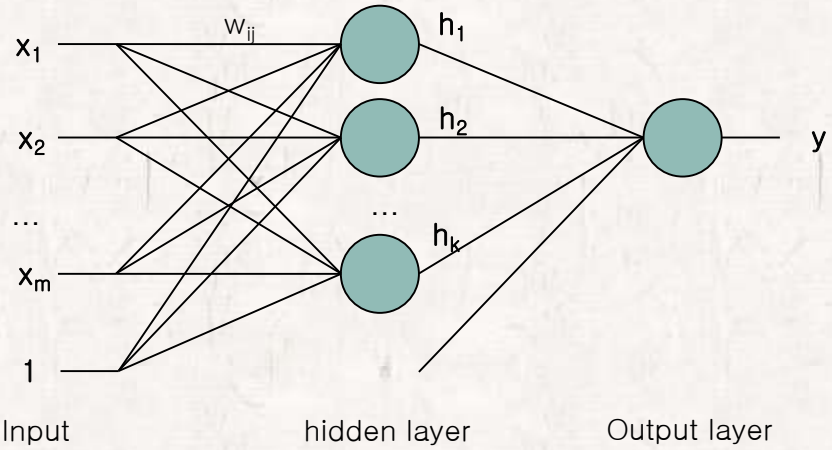
- I have a data set

$$(x_{11}, x_{12}, \ldots, x_{1m}, y_1)$$
$$(x_{21}, x_{22}, \ldots, x_{2m}, y_2)$$
$$\ldots$$
$$(x_{n1}, x_{n2}, \ldots, x_{nm}, y_n)$$

- I want to build a Neural Network which generalizes the data
  - Step 1: determine the structure of neural network
    - # of layers, # of nodes in each layer
  - Step 2: determine the weights of links
    - How???

Learning Algorithm

- Step 1:



- Step 2:
  - How many weights? (m+1)*k+(k+1)
  - How?
    - Define an error function
    - Find weights which minimize the error

Learning Algorithm

- Error Function

$$E(\mathbf{w}) = \frac{1}{2}\sum_{i=1}^{n}\left(N(\mathbf{w},\mathbf{x}_i) - y_i\right)^2$$

  - $N(\mathbf{w},\mathbf{x})$: The output of a neural network
  - y_i : Target value of $\mathbf{x}_i$

$(x_{11}, x_{12}, \ldots, x_{1d}, y_1)$
$(x_{21}, x_{22}, \ldots, x_{2d}, y_2)$
$\ldots$
$(x_{n1}, x_{n2}, \ldots, x_{nd}, y_n)$

Error Back Propagation

- Basic Idea
  - Given input-target pairs and output of NN

$D_1=(x_{11}, x_{12}, \ldots, x_{1d}, t_{11}, t_{12}, \ldots, t_{1m})$
$D_2=(x_{21}, x_{22}, \ldots, x_{2d}, t_{21}, t_{22}, \ldots, t_{2m})$
$\ldots$
$D_N=(x_{N1}, x_{N2}, \ldots, x_{Nd}, t_{N1}, t_{N2}, \ldots, t_{Nm})$

inputs targets

$(o_{11}, o_{12}, \ldots, o_{1m})$
$(o_{21}, o_{22}, \ldots, o_{2m})$
$\ldots$
$(o_{N1}, o_{N2}, \ldots, o_{Nm})$

Outputs of NN

  - Minimize the error

$$E(\mathbf{w}) = \sum_{n=1}^{N} E_n(\mathbf{w}) \quad \text{where} \quad E_n(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{m}(t_{nk} - o_{nk})^2$$

Error Back Propagation

- Basic Idea
  - Remember

$$\frac{\partial}{\partial w}E = \sum_{n=1}^{N}\frac{\partial}{\partial w}E_n \qquad \text{because} \qquad E = \sum_{n=1}^{N}E_n$$

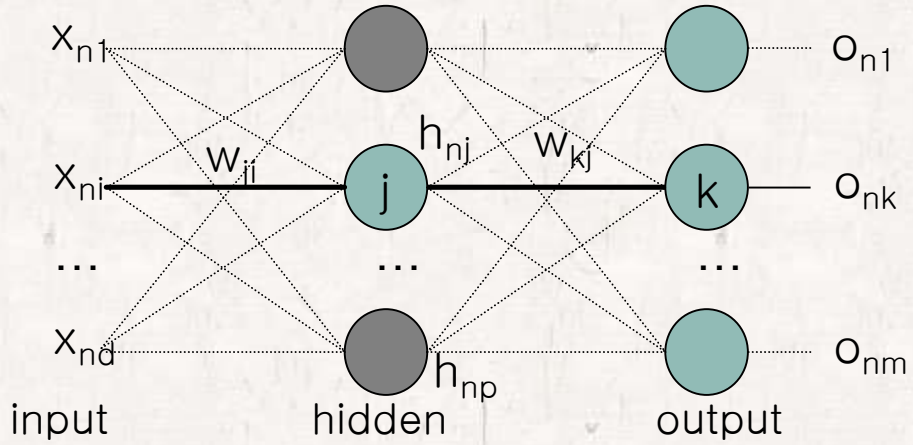
  - So, we need to evaluate: $\frac{\partial}{\partial w}E_n$

Error Back Propagation

- Remind Error Function

$D_n=(x_{n1}, x_{n2}, \ldots, x_{nd}, t_{n1}, t_{n2}, \ldots, t_{nm})$

$(o_{n1}, o_{n2}, \ldots, o_{nm})$



$$O_{nk} = \frac{1}{1+\exp\left(-\left(w_{k0}+\sum_{j=1}^{p} w_{kj} h_{nj}\right)\right)}$$

$$h_{nj} = \frac{1}{1+\exp\left(-\left(w_{i0}+\sum_{i=1}^{d} w_{ji} x_{ni}\right)\right)}$$

$$E_n(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{m}(t_{nk}-o_{nk})^2$$

So, you can evaluate

$$\frac{\partial}{\partial w_{ji}} E_n \quad \text{and} \quad \frac{\partial}{\partial w_{kj}} E_n$$

$$E_n(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{m}\left(t_{nk} - \frac{1}{1+\exp\left(-\left(w_{k0}+\sum_{i=1}^{p} w_{kj}\left(\frac{1}{1+\exp\left(-\left(w_{i0}+\sum_{i=1}^{d} w_{ji} x_{ni}\right)\right)}\right)\right)\right)}\right)^2$$

Error Back Propagation

- Remind Error Function
  - Too complex to differentiate

$$E_n(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{m}\left(t_{nk} - \frac{1}{1+\exp\left(-\left(w_{k0} + \sum_{i=1}^{p} w_{kj}\left(\frac{1}{1+\exp\left(-\left(w_{i0} + \sum_{i=1}^{d} w_{ji}x_{ni}\right)\right)}\right)\right)\right)}\right)^2$$

  - Let's use chain rule
    - Case 1: when w is between output and hidden layer
    - Case 2: when w is between hidden and input layer

Error Back Propagation

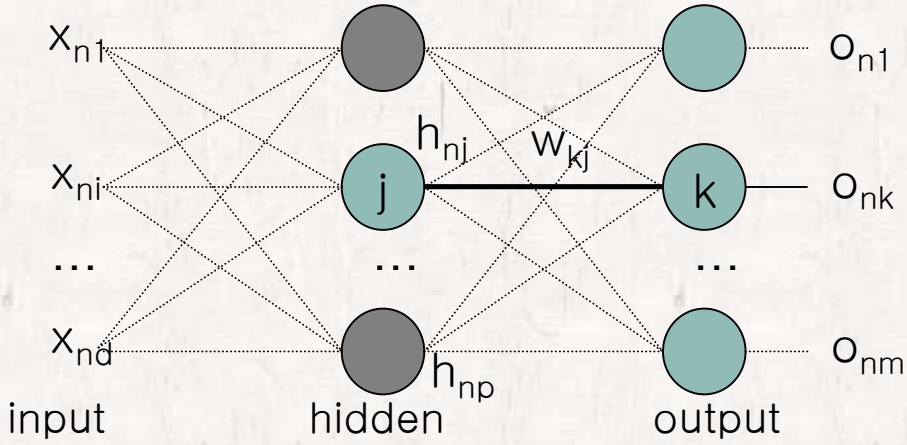
- Case 1: Weights between output and hidden layer
  - For $D_n=(x_{n1}, x_{n2}, \ldots, x_{nd}, t_{n1}, t_{n2}, \ldots, t_{nm})$

$$h_{nj} = \text{output of j-th node in hidden layer given } \mathbf{x}_n$$

$$net_{nk} = h_{n1}w_{k1} + h_{n2}w_{k2} + \cdots + h_{np}w_{kp}$$

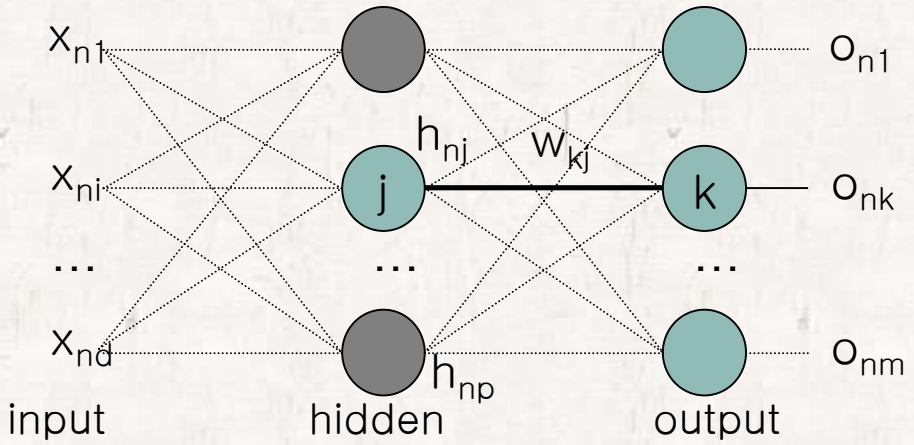
$$o_{nk} = \text{sigmoid}(\, net_{nk})$$

$$E_n(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{m}(t_{nk} - o_{nk})^2$$



Error Back Propagation

- Case 1: Weights between output and hidden layer



$$\Delta w_{kj} = -\eta \frac{\partial E_n}{\partial w_{kj}}$$

$$net_{nk} = h_{n1}w_{k1} + h_{n2}w_{k2} + \cdots + h_{np}w_{kp}$$

$$o_{nk} = \text{sigmoid}(\, net_{nk})$$

$$E_n(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{m}(t_{nk} - o_{nk})^2$$

$$\frac{\partial E_n}{\partial w_{kj}} = \frac{\partial E_n}{\partial net_{nk}} \frac{\partial net_{nk}}{\partial w_{kj}}$$

$$= \frac{\partial E_n}{\partial o_{nk}} \frac{\partial o_{nk}}{\partial net_{nk}} \frac{\partial net_{nk}}{\partial w_{kj}}$$

$$= \frac{\partial E_n}{\partial o_{nk}} \frac{\partial o_{nk}}{\partial net_{nk}} h_{nj}$$

Error Back Propagation

- Case 1: Weights between output and hidden layer –con'd

$$\frac{\partial E_n}{\partial w_{kj}} = \frac{\partial E_n}{\partial o_{nk}} \frac{\partial o_{nk}}{\partial net_{nk}} h_{nj}$$

$$\frac{\partial o_{nk}}{\partial net_{nk}} = \frac{\partial \,\mathrm{sigmoid}(\, net_{nk})}{\partial net_{nk}} = o_{nk}(1-o_{nk})$$

$$\frac{\partial E_n}{\partial o_{nk}} = \frac{1}{2}\sum_{q=1}^{m}(t_{nq}-o_{nq})^2 = \frac{\partial}{\partial o_{nk}}\frac{1}{2}(t_{nk}-o_{nk})^2 = \frac{1}{2}2(t_{nk}-o_{nk})\frac{\partial(t_{nk}-o_{nk})}{\partial o_{nk}} = -(t_{nk}-o_{nk})$$

$$\frac{\partial E_n}{\partial w_{kj}} = -(t_{nk}-o_{nk})o_{nk}(1-o_{nk})h_{nj}$$

Error Back Propagation

- Case 1: Weights between output and hidden layer –con'd

$$\frac{\partial E_n}{\partial w_{kj}} = -(t_{nk} - o_{nk}) o_{nk} (1 - o_{nk}) h_{nj}$$

$$\frac{\partial E}{\partial w_{kj}} = \sum_{n=1}^{N} \frac{\partial E_n}{\partial w_{kj}} = -\sum_{n=1}^{N} (t_{nk} - o_{nk}) o_{nk} (1 - o_{nk}) h_{nj}$$

$$\Delta w_{kj} = -\eta \frac{\partial E}{\partial w_{kj}} = \eta \sum_{n=1}^{N} (t_{nk} - o_{nk}) o_{nk} (1 - o_{nk}) h_{nj}$$

Error Back Propagation

- Case 2: Weights between hidden and input layer
  - For $D_n=(x_{n1}, x_{n2}, \ldots, x_{nd}, t_{n1}, t_{n2}, \ldots, t_{nm})$

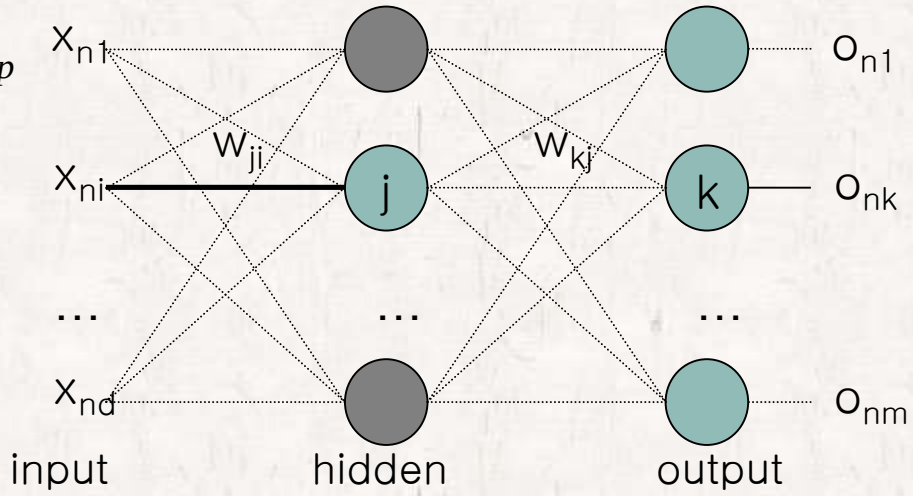
$$net_{nj} = x_{n1}w_{j1} + x_{n2}w_{j2} + \cdots + x_{nd}w_{jd}$$

$$h_{nj} = \text{sigmoid}(\ net_{nj})$$

$$net_{nk} = h_{n1}w_{k1} + h_{n2}w_{k2} + \cdots + h_{np}w_{kp}$$

$$o_{nk} = \text{sigmoid}(\ net_{nk})$$

$$E_n(\mathbf{w}) = \frac{1}{2}\sum_{k=1}^{m}(t_{nk} - o_{nk})^2$$



Error Back Propagation

- Case 2: Internal weights – con'd

$$\frac{\partial E_n}{\partial w_{ji}} = \frac{\partial}{\partial w_{ji}} \frac{1}{2} \sum_{k=1}^{m} (t_{nk} - o_{nk})^2$$

$$= \frac{1}{2} \sum_{k=1}^{m} \frac{\partial}{\partial w_{ji}} (t_{nk} - o_{nk})^2$$

$$= \frac{1}{2} \sum_{k=1}^{m} \frac{\partial (t_{nk} - o_{nk})^2}{\partial o_{nk}} \frac{\partial o_{nk}}{\partial net_{nk}} \frac{\partial net_{nk}}{\partial h_{nj}} \frac{\partial h_{nj}}{\partial net_{nj}} \frac{\partial net_{nj}}{\partial w_{ji}}$$

$$= \frac{1}{2} \sum_{k=1}^{m} -2(t_{nk} - o_{nk}) \cdot o_{nk}(1 - o_{nk}) \cdot w_{kj} \cdot h_{nj}(1 - h_{nj}) \cdot x_{ni}$$

$$= -h_{nj}(1 - h_{nj}) x_{ni} \sum_{k=1}^{m} w_{kj} (t_{nk} - o_{nk}) o_{nk}(1 - o_{nk})$$

$$\frac{\partial (t_{nk} - o_{nk})^2}{\partial o_{nk}} = -2(t_{nk} - o_{nk})$$

$$\frac{\partial o_{nk}}{\partial net_{nk}} = o_{nk}(1 - o_{nk})$$

$$\frac{\partial net_{nk}}{\partial h_{nj}} = w_{kj}$$

$$\frac{\partial h_{nj}}{\partial net_{nj}} = h_{nj}(1 - h_{nj})$$

$$\frac{\partial net_{nj}}{\partial w_{ji}} = x_{ni}$$

Error Back Propagation

- Case 2: Internal weights – con'd

$$\frac{\partial E_n}{\partial w_{ji}} = -x_{ni} h_{nj} (1 - h_{nj}) \sum_{k=1}^{m} w_{kj} (t_{nk} - o_{nk}) o_{nk} (1 - o_{nk})$$

$$\frac{\partial E}{\partial w_{ji}} = \sum_{n=1}^{N} \frac{\partial E_n}{\partial w_{ji}} = \sum_{n=1}^{N} \left(-x_{ni} h_{nj} (1 - h_{nj}) \sum_{k=1}^{m} w_{kj} (t_{nk} - o_{nk}) o_{nk} (1 - o_{nk}) \right)$$

$$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}} = \eta \sum_{n=1}^{N} \left(x_{ni} h_{nj} (1 - h_{nj}) \sum_{k=1}^{m} w_{kj} (t_{nk} - o_{nk}) o_{nk} (1 - o_{nk}) \right)$$

Error Back Propagation

initialize all weights to small random numbers

Repeat

for each output node's weight w_{kj} $\Delta w_{kj} = 0$

for each hidden node's weight w_{ji} $\Delta w_{ji} = 0$

for each training example $D_n = <x_{n1}, x_{n2}, \ldots, x_{nd}, t_{n1}, t_{n2}, \ldots, t_{nm}>$

for each hidden node h_j $h_{nj} \leftarrow \text{sigmoid}\left(\sum_{i=1}^{d} w_{ji} x_{ni}\right)$

for each output node o_k $o_{nk} \leftarrow \text{sigmoid}\left(\sum_{j=1}^{p} w_{kj} h_{nj}\right)$

for each output node's weight w_{kj} $\Delta w_{kj} += \eta (t_{nk} - o_{nk}) o_{nk} (1 - o_{nk}) h_{nj}$

for each hidden node's weight w_{ji} $\Delta w_{ji} += \eta h_{nj} (1 - h_{nj}) x_{ni} \sum_{k=1}^{m} o_{nk} (1 - o_{nk}) (t_k - o_{nk}) w_{kj}$

end of for

for each hidden node's weight $w_{kj} \leftarrow w_{kj} + \Delta w_{kj}$

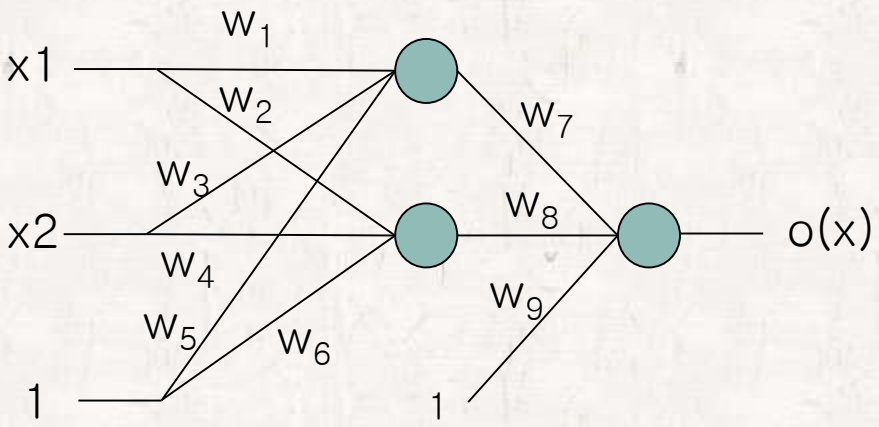
for each output node's weight $w_{ji} \leftarrow w_{ji} + \Delta w_{ji}$

until (termination condition)

Error Back Propagation

- Example : XOR
  - Hidden nodes : 2
  - Learning rate : 0.5

x1	x2	y
1	1	0
1	0	1
0	1	1
0	0	0

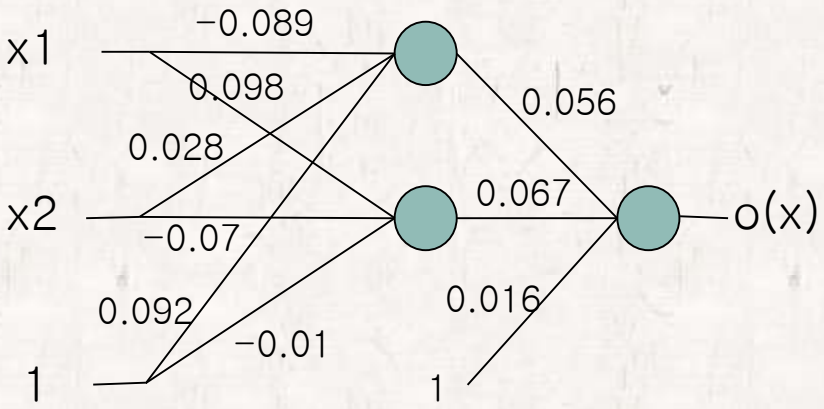


Error Back Propagation

- Example : XOR

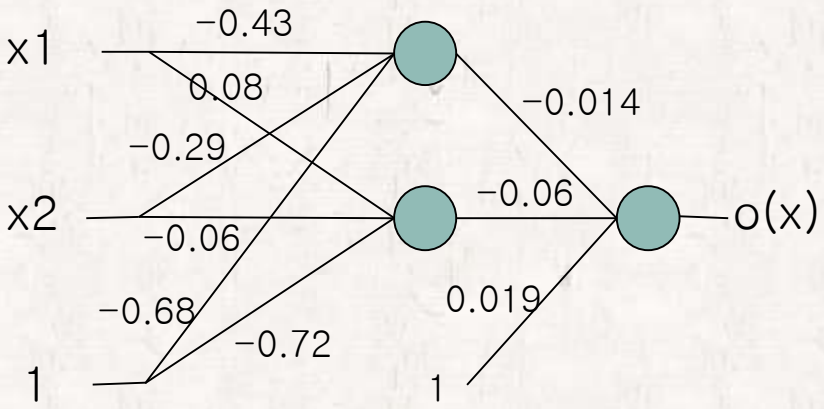
Iteration : 0

x1	x2	y	o
1	1	0	0.52
1	0	1	0.50
0	1	1	0.52
0	0	0	0.55



Iteration : 1000

x1	x2	y	o
1	1	0	0.50
1	0	1	0.48
0	1	1	0.50
0	0	0	0.52

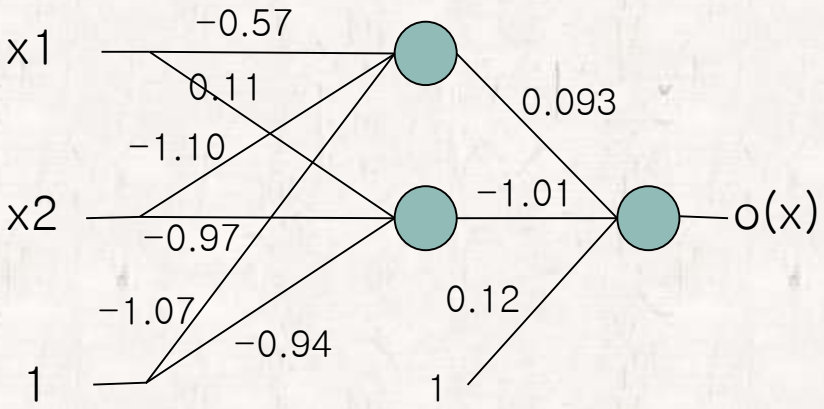


Error Back Propagation

- Example : XOR

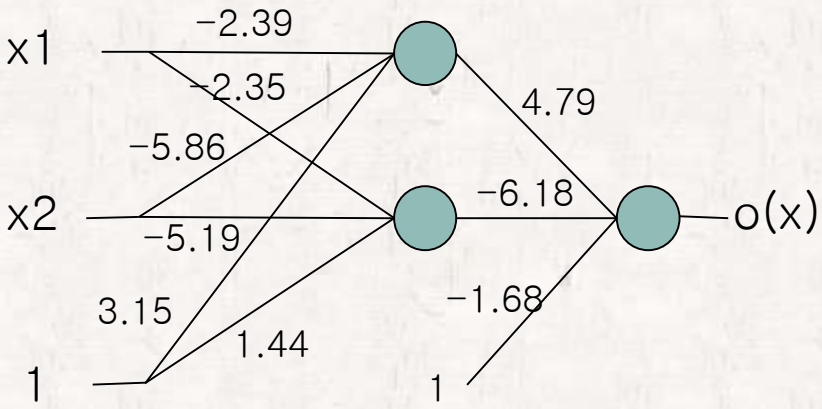
Iteration : 2000

x1	x2	y	o
1	1	0	0.53
1	0	1	0.48
0	1	1	0.50
0	0	0	0.48



Iteration : 3000

x1	x2	y	o
1	1	0	0.30
1	0	1	0.81
0	1	1	0.81
0	0	0	0.11

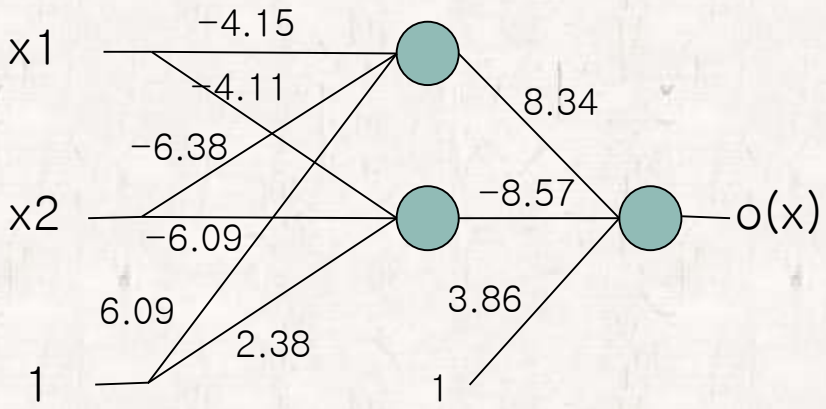


Error Back Propagation

- Example : XOR

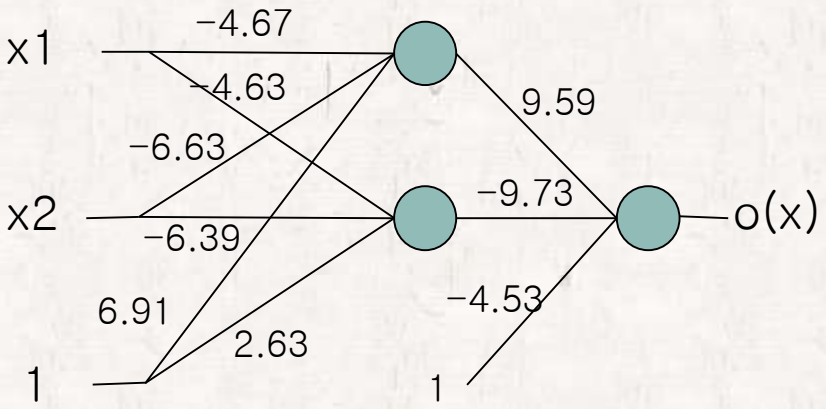
Iteration : 5000

x1	x2	y	o
1	1	0	0.05
1	0	1	0.96
0	1	1	0.96
0	0	0	0.03



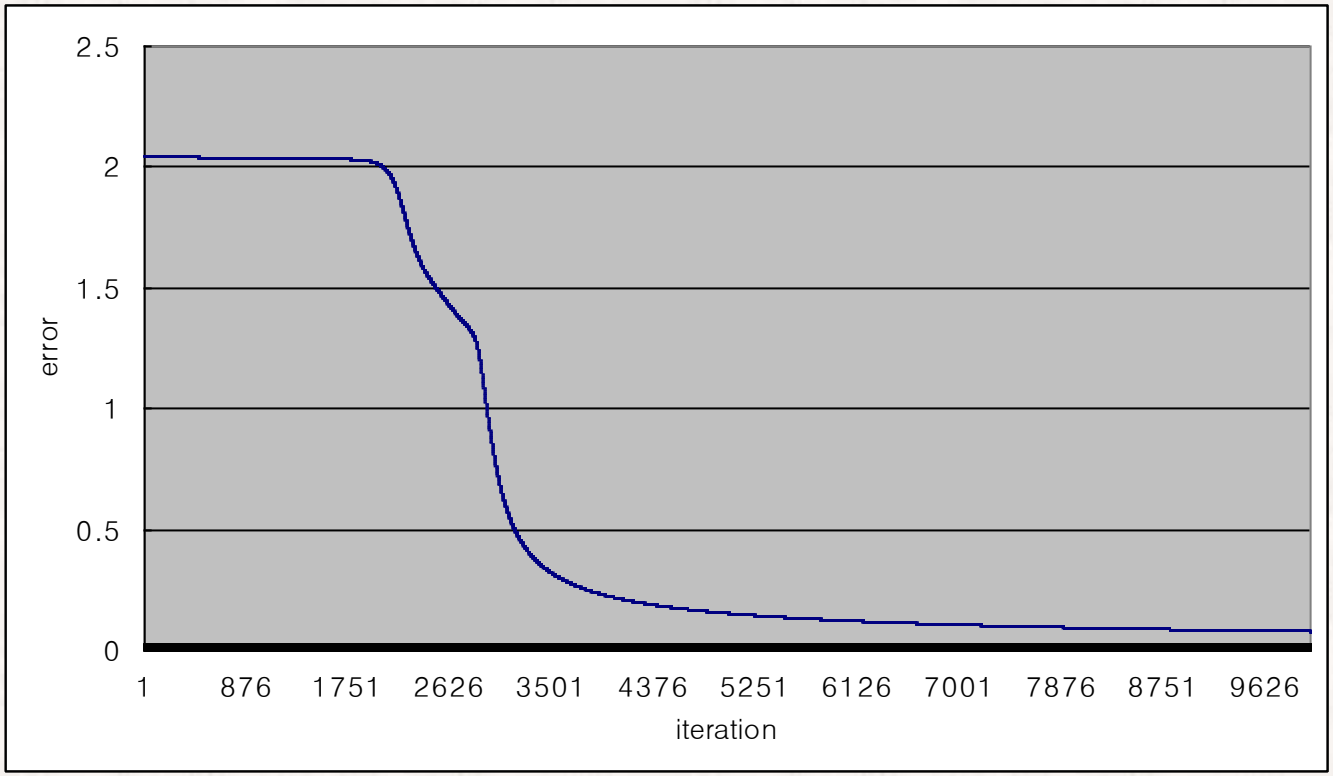
Iteration : 10000

x1	x2	y	o
1	1	0	0.02
1	0	1	0.98
0	1	1	0.98
0	0	0	0.02



Error Back Propagation

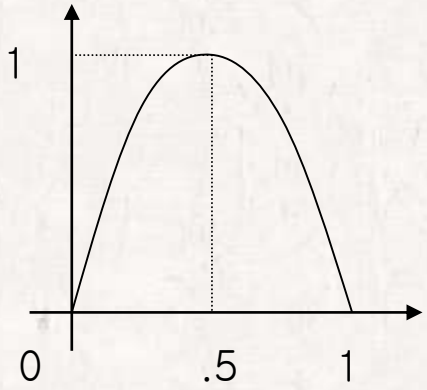
- Example : XOR
  - Error graph

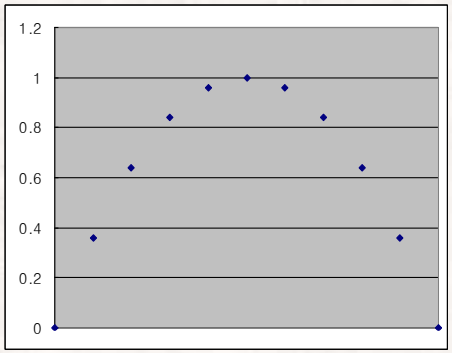


Error Back Propagation

- Example2 :
  - Hidden nodes : 4
  - Iteration : 500,000
  - Learning rate : 0.7

$$f(x) = 4x*(1-x)$$

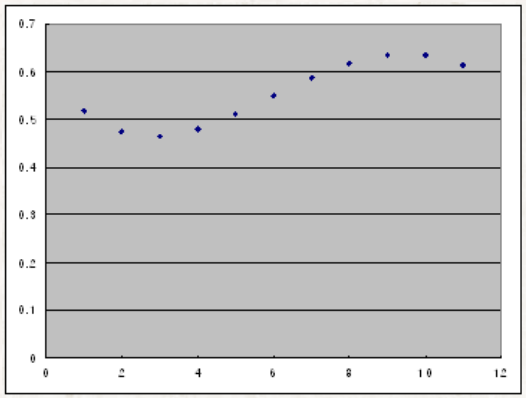
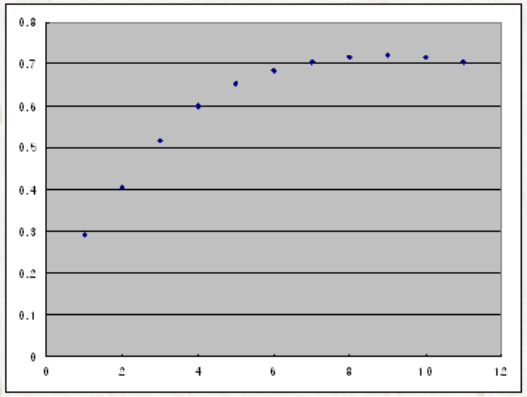
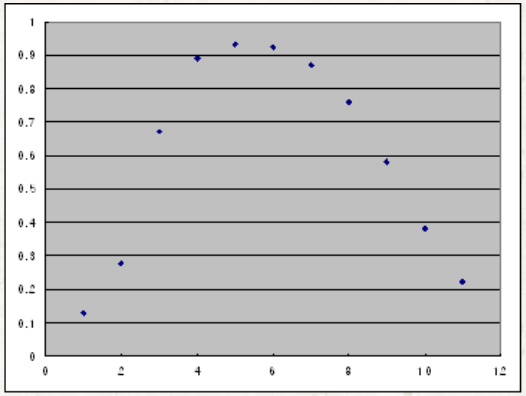
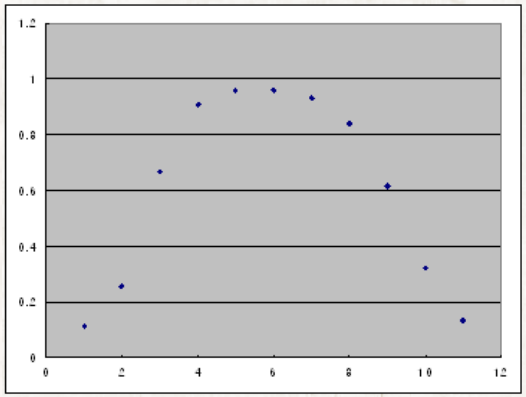
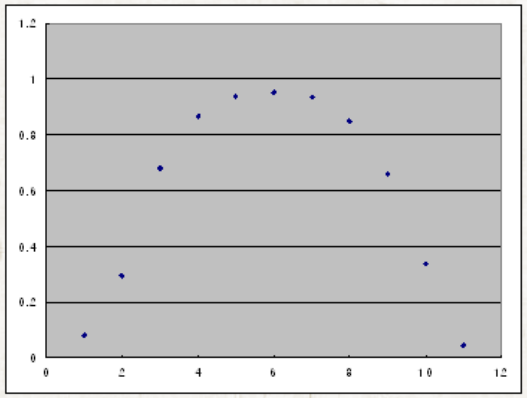
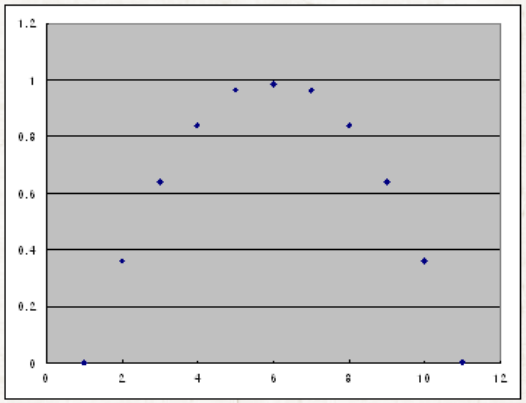




Input	Output
0.00	0.00
0.10	0.36
0.20	0.64
0.30	0.84
0.40	0.96
0.50	1.00
0.60	0.96
0.70	0.84
0.80	0.64
0.90	0.36
1.00	0.00

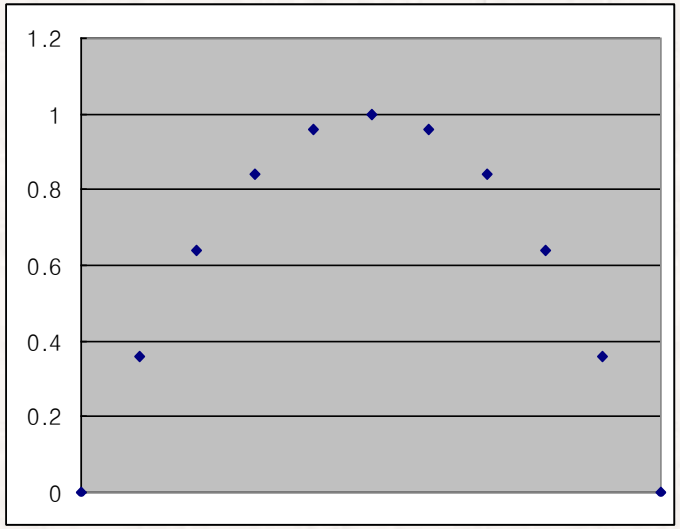
Error Back Propagation

- Example2

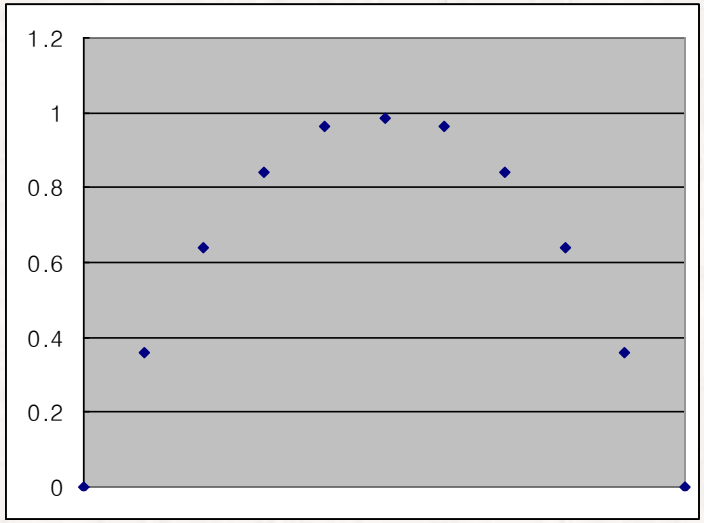







Error Back Propagation

- We gave only 11 points
  - A NN learned only that 11 points



Training data

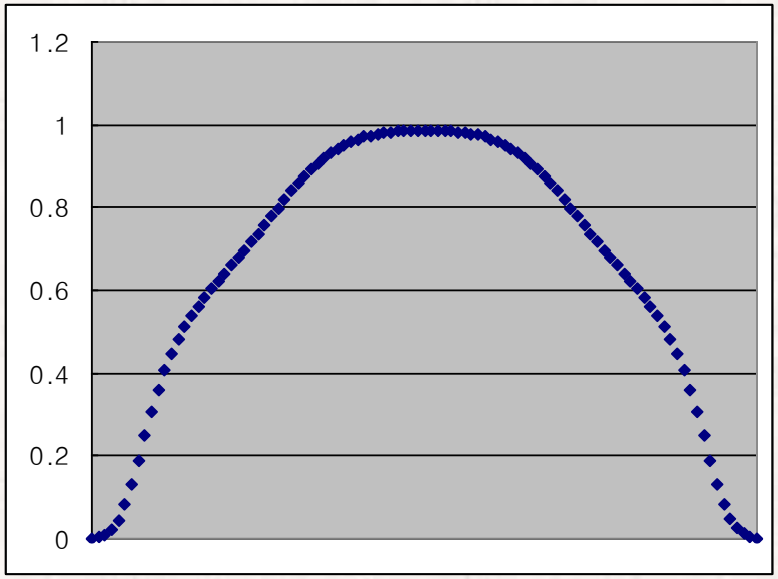


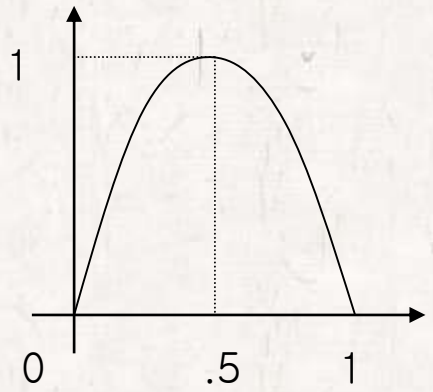
Training result

  - Can the NN answer to the un-learned points?

Error Back Propagation

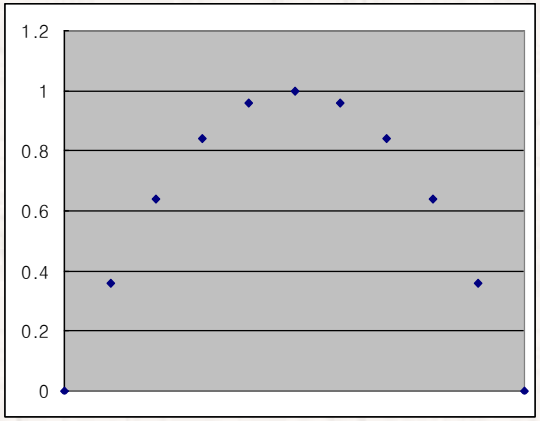
- Yes, NNs generalize what they have learned



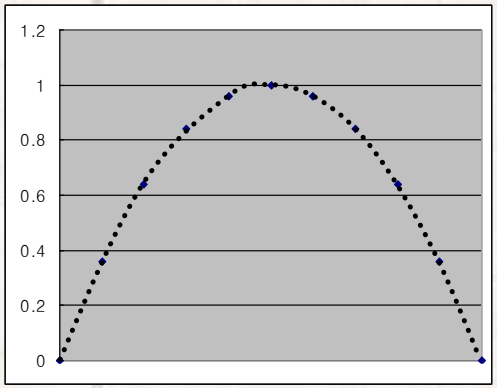


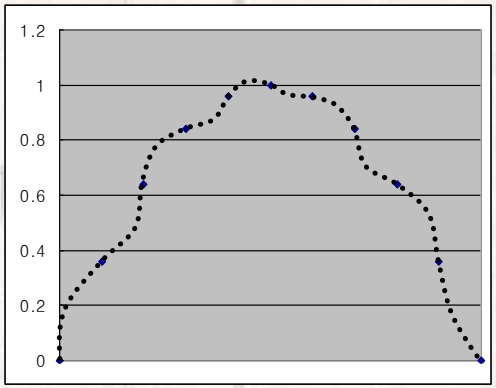
Generalization, Overfitting

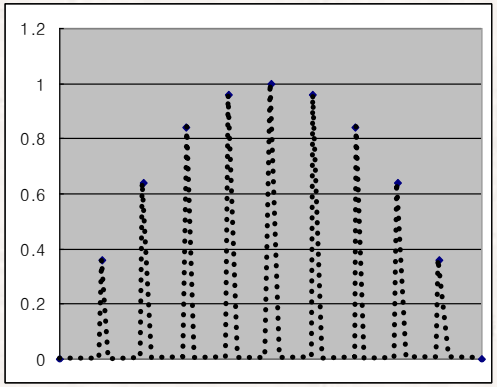
- Which one is better?



Training data







How to Control??

Summary

- A perceptron will find a hyperplane (straight line in 1 dimension) that minimizes the error between the target and the output
- Two-layer perceptrons will tend to find a piece-wise hyperplane that minimizes the error between the target and the output
- Neural networks smoothly interpolate the training data points.
- Very efficient in learning data
- No generation of human-readable knowledge

Discussion

- Two class classification
  - Use 0 and 1 for class labels
  - Use one perceptron at the output layer
  - Prediction

$$class(\mathbf{x}) = \begin{cases} 0 & N(\mathbf{x}) > 0.5 \\ 1 & \text{O.W.} \end{cases}$$

- *n* class classification
  - Use 1 to *n* for class labels
  - Use *n* perceptrons at the output layer
  - Prediction

$$class(\mathbf{x}) = \arg\max_i N_i(\mathbf{x})$$

Why not?
- Use 1 to n for class labels
- Use one perceptron
- Prediction

$$class(\mathbf{x}) = i$$

$$\text{if } \frac{(i-1)}{n} \le N(\mathbf{x}) < \frac{i}{n}$$

Discussion

- What if you have categorical inputs
  - Two inputs

$$x_1 \in R$$
$$x_2 \in \{\text{Yellow, Red, Blue}\}$$

  - Create a new input variable for each categorical value

$$x_2 = \begin{cases} 1 & \text{if original } x_2 \text{ is Yellow} \\ 0 & \text{Otherwise} \end{cases}$$

$$x_3 = \begin{cases} 1 & \text{if original } x_2 \text{ is Red} \\ 0 & \text{Otherwise} \end{cases}$$

$$x_4 = \begin{cases} 1 & \text{if original } x_2 \text{ is Blue} \\ 0 & \text{Otherwise} \end{cases}$$

$$(0.1, \text{Red}) \quad (0.2, \text{Blue}) \quad (0.3, \text{Yellow}) \quad (0.4, \text{Red}) \Rightarrow (0.1, 0,1,0) \quad (0.2, 0,0,1) \quad (0.3, 1,0,0) \quad (0.4, 0,1,0)$$

Discussion

- For Regression
  - Normalize the outputs into [0,1]
    - Why?
  - Or, use a linear output node

